

RESEARCH INTERNSHIP PROJECT REPORT

**RF-AWARE V2V COMMUNICATION AND
ADAPTIVE VEHICLE CONTROL**

Scene-Conditioned Software-in-the-Loop Co-Simulation using Deep Learning

Submitted in partial fulfilment of the academic requirements for the degree of

BACHELOR OF TECHNOLOGY

Electronics and Communication Engineering

by

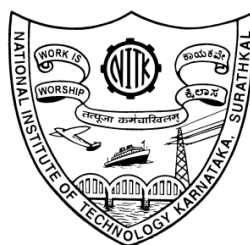
PRAJWAL HALAPETI (231EC236)

Under the guidance of

PRABHU KRISHNAN

Research Mentor

Department of Electronics and Communication Engineering



Department of Electronics and Communication Engineering

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL

Surathkal, Mangaluru – 575025

August 2026

Certificate

This is to certify that the Research Internship Project entitled *RF-Aware V2V Communication and Adaptive Vehicle Control*, submitted by **PRAJWAL HALAPETI** (231EC236), is a bona fide record of the work carried out under the guidance of **PRABHU KRISHNAN**, Research Mentor, Department of Electronics and Communication Engineering, National Institute of Technology Karnataka, Surathkal. The work documented in this report represents the implementation and evaluation performed during the project period.

PRAJWAL HALAPETI

231EC236

Bachelor of Technology

Department of Electronics and Communication Engineering

NITK Surathkal – 575025

PRABHU KRISHNAN

Research Mentor

Department of Electronics and Communication Engineering

NITK Surathkal – 575025

(Signature with Date and Seal)

(Student Signature with Date)

Abstract

This project presents a scene-conditioned software-in-the-loop framework for Vehicle-to-Vehicle (V2V) communication and adaptive longitudinal vehicle control. A dense urban driving scenario was implemented in MATLAB/Simulink with dynamic lead-vehicle tracking, route-based RF conditioning, and closed-loop speed control. A Python/PyTorch inference service processes in-phase and quadrature (I/Q) frames and estimates NLOS probability, transmitter identity, signal-to-noise ratio, packet-error risk, delay spread, and an aggregate RF-quality score. MATLAB and Python exchange structured requests and responses through ZeroMQ.

The cleaned synthetic dataset contains 33,268 finite I/Q frames across three transmitter classes. Post-training evaluation produced an NLOS F1-score of 96.69%, balanced accuracy of 96.81%, ROC-AUC of 99.99%, and median/p95 single-frame inference latencies of 1.85/2.26 ms. The RF experiment was calibrated to the ego-route interval 959.172–1073.437 m. In the final 270-second clean and NLOS closed-loop runs, the MATLAB–Python software link remained successful for 100% of logged samples. Within the calibrated zone, mean RF quality changed from 0.7959 in the clean run to 0.5343 under NLOS, while mean predicted NLOS probability changed from 0.1330 to 0.3093. The physical gap/TTC safety logic remained independent of the RF layer, and RF adaptation was enabled only inside the calibrated zone to preserve scenario timing elsewhere. The implementation is intentionally described as a **scene-conditioned RF replay/co-simulation**; it is not a geometry-based electromagnetic ray-tracing digital twin.

Keywords: V2V communication, NLOS detection, RF intelligence, deep learning, adaptive longitudinal control, MATLAB/Simulink, PyTorch, ZeroMQ, software-in-the-loop.

Contents

Certificate	i
Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope and Main Contributions	2
2 Background and Related Work	3
2.1 V2V Communication and Longitudinal Control	3
2.2 RF Learning from I/Q Samples	3
2.3 V2X Security Context	3
2.4 Deep Learning for Wireless Signals	4
2.5 Design Position	4
3 System Architecture	5
3.1 End-to-End Data Flow	5
3.2 Final MATLAB/Simulink Implementation	5
3.3 Key Signal Interface	6
3.4 Python RF Inference Layer	6
3.5 Fail-Safe Interface Behavior	6
4 Dataset and Deep-Learning RF Model	7
4.1 Finite Synthetic V2V I/Q Dataset	7
4.2 Preprocessing and Outputs	7
4.3 Final Training Configuration	7
4.4 Post-Training Evaluation	7
5 Scene-Conditioned SIL Implementation	9
5.1 Driving Scenario and RF Interaction Region	9
5.2 RF-Zone Calibration	10
5.3 Conditioned I/Q Selection	10
5.4 ZeroMQ Software Bridge	10
5.5 RF-Zone Adaptive Gate	11
5.6 Final Experiment Configuration	11

6	Experiments, Results and Reliability	12
6.1	Validation Sequence	12
6.2	RF-Only NLOS Validation	12
6.3	Final Clean-versus-NLOS Closed-Loop Comparison	13
6.4	Security and Reliability Assessment	14
7	Conclusion and Future Work	15
7.1	Conclusion	15
7.2	Future Work	15
A	Reproduction Commands	16

List of Figures

3.1	End-to-end data flow of the scene-conditioned V2V RF software-in-the-loop platform.	5
3.2	Final Simulink implementation with route-based RF context and RF-zone adaptive gating.	6
5.1	Bird’s-Eye Scope during the highway segment, showing vehicle- and world-coordinate views.	9
5.2	Bird’s-Eye Scope in the dense roundabout used as the final RF interaction region.	9
5.3	Scene-conditioning calibration and valid-lead audit for the selected RF zone. . .	10
5.4	Verified MATLAB–Python communication with 100% smoke-test link success and nonconstant RF outputs.	11
6.1	Command-window audit of the full RF-only NLOS run.	12
6.2	Final corrected clean-versus-NLOS comparison aligned by ego-route position. .	13
6.3	Command-window audit of the final corrected clean and NLOS runs.	13

Chapter 1

Introduction

1.1 Motivation

Vehicle-to-Vehicle communication can extend a vehicle's awareness beyond local sensors by sharing state, warning, and intent information with nearby vehicles. This can support cooperative braking, safer car-following, and more informed longitudinal control. The benefit, however, depends on whether the wireless information is timely and trustworthy. Urban blockage, multipath propagation, interference, packet errors, or manipulated transmissions can reduce the value of V2V data even when the physical driving scene itself is unchanged.

The engineering objective of this work is therefore not simply to classify an RF signal. It is to connect RF intelligence to a realistic closed-loop driving simulation while preserving the physical safety controller as the dominant layer. The project combines vehicle dynamics, dynamic lead tracking, scene-conditioned RF events, deep-learning inference on raw I/Q frames, a MATLAB–Python software interface, and RF-aware longitudinal adaptation in one reproducible experiment platform.

1.2 Problem Statement

The project addresses the following problem:

How can an urban driving simulation incorporate real-time RF inference and scene-dependent NLOS conditions into adaptive longitudinal control without disturbing the established physical safety behavior outside the intended RF experiment region?

The implementation must satisfy four practical requirements:

1. **Physical Consistency:** preserve the established ego trajectory and traffic interactions outside the RF experiment region.
2. **Low-Latency RF Inference:** convert raw I/Q frames into useful communication-quality metrics fast enough for software-in-the-loop operation.
3. **Reliable SIL Communication:** maintain a robust MATLAB–Python request/response interface during long simulations.
4. **Fail-Safe Control:** allow RF quality to increase controller conservatism without overriding physical gap and time-to-collision safety logic.

1.3 Objectives

The final project objectives are:

- build a repeatable urban V2V scenario in MATLAB/Simulink with dynamic lead-vehicle tracking;
- train a multi-task deep-learning model on finite synthetic I/Q data for NLOS and RF-property estimation;
- connect Python RF inference to Simulink through a robust ZeroMQ interface;
- activate NLOS experiments from the actual ego-route position rather than from a fixed simulation-time window;
- compare clean and NLOS closed-loop runs using identical scenario, duration, run seed, speed limit, and controller settings; and
- report both strong results and model limitations without treating unvalidated outputs as safety guarantees.

1.4 Scope and Main Contributions

The completed system is a **scene-conditioned RF replay/co-simulation**. The physical scene determines when the RF experiment is active and the current valid lead determines the V2V communication peer. A dataset-backed Python service then selects and processes corresponding clean or NLOS I/Q frames. Buildings are used to define a meaningful urban interaction region, but the RF values are not computed by electromagnetic ray tracing.

The main contributions are:

1. an end-to-end MATLAB/Simulink–Python V2V SIL platform combining traffic dynamics, RF inference, and longitudinal control;
2. a route-calibrated RF context using `path_s`, lead ActorID, lead validity, and lead gap;
3. a robust ZeroMQ interface with explicit timeouts, socket recovery, variable-length JSON handling, and conservative fallback outputs;
4. a zone-gated adaptive-control path that preserves earlier physical traffic interactions; and
5. a reproducible clean-versus-NLOS comparison with quantitative RF, latency, and closed-loop evidence.

Chapter 2

Background and Related Work

2.1 V2V Communication and Longitudinal Control

A longitudinal controller regulates ego-vehicle acceleration from variables such as ego speed, lead speed, separation gap, relative speed, and time-to-collision. V2V information can supplement these local measurements with communication-aware context. In a safety-oriented design, however, wireless confidence should modify supervisory behavior such as headway or trust; it should not replace emergency kinematic braking.

This project follows that layered principle. Physical lead tracking and low-TTC logic remain continuously available. RF quality, NLOS probability, packet-error risk, and age of information are supervisory inputs that can make the controller more conservative only when RF adaptation is enabled.

2.2 RF Learning from I/Q Samples

ORACLE demonstrated that convolutional neural networks can identify radios directly from raw I/Q samples by learning transmitter-specific hardware variations, rather than relying on identifiers such as MAC addresses [1]. The work also emphasizes a core difficulty for RF fingerprinting: propagation-channel effects can dominate the received waveform and reduce device separability across environments.

The present project uses the same broad idea—learning useful properties directly from I/Q observations—but treats transmitter identity as only one output. The model also estimates NLOS probability and continuous RF quantities that can be converted into a communication-quality signal for vehicle control. Because transmitter classification remained substantially weaker than NLOS detection in the final evaluation, device identity is retained as an auxiliary diagnostic and is not used as a hard authentication gate.

2.3 V2X Security Context

Hasan *et al.* survey the V2X ecosystem, its security and privacy threats, standardization activities, and defensive mechanisms [2]. The survey highlights the safety consequence of false or unavailable vehicular information and covers threats including jamming, spoofing, replay, flooding, and Sybil behavior.

The experiment service developed here includes condition hooks for NLOS, jamming, and spoofing. The final reported validation is deliberately narrower: clean and NLOS conditions are the two conditions used for the final closed-loop comparison. This separation keeps implemented capability distinct from experimentally validated evidence.

2.4 Deep Learning for Wireless Signals

Deep learning can learn representations directly from high-dimensional RF samples and has been applied to physical-layer inference tasks such as modulation recognition and learned communication systems [3]. The practical risks are distribution shift, class imbalance, numerical instability, and evaluation leakage. In this work, invalid frames are removed before training, every online frame is normalized and checked for finite values, and model limitations are reported explicitly.

2.5 Design Position

The final platform joins three normally separate components: (1) a physical driving scenario, (2) a learned RF inference service, and (3) a longitudinal controller. The driving scene creates the communication context; the RF model converts I/Q frames into link intelligence; and the controller uses that information without weakening the physical safety layer. This integration is the central engineering contribution of the project.

Chapter 3

System Architecture

3.1 End-to-End Data Flow

The complete implementation is organized as a simple vertical processing chain so that the relationship between the driving scene, RF intelligence, and vehicle control is explicit.

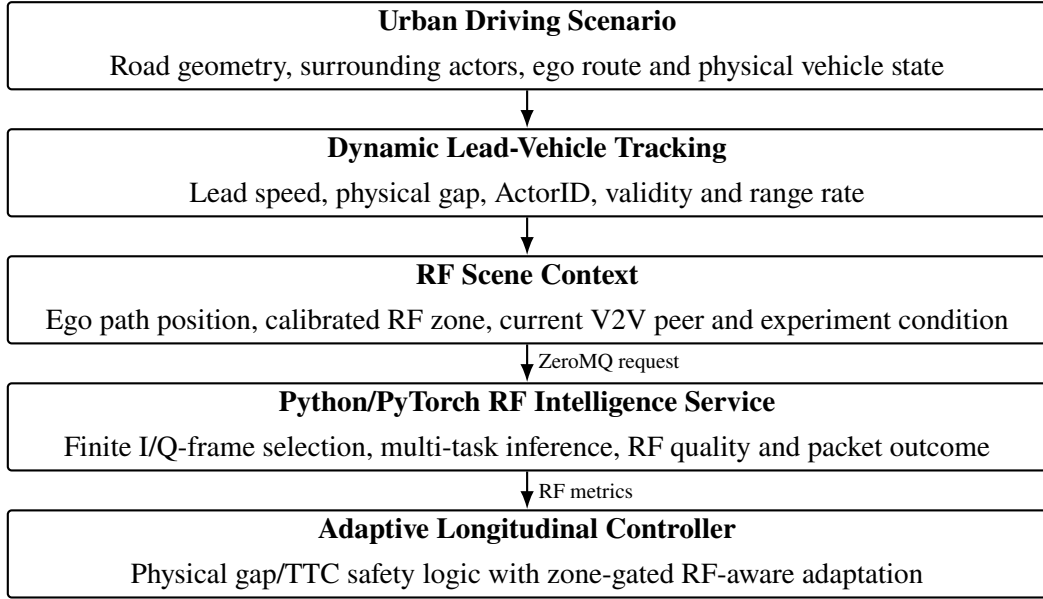


Figure 3.1: End-to-end data flow of the scene-conditioned V2V RF software-in-the-loop platform.

3.2 Final MATLAB/Simulink Implementation

The final model contains the Scenario Reader, ego-path generation, dynamic lead tracker, packet counter, RF Scene Context, RF metrics receiver, RF-zone adaptive gate, and longitudinal controller. Physical scene signals are reused to establish RF context, preventing the RF experiment from evolving independently of the driving scenario.

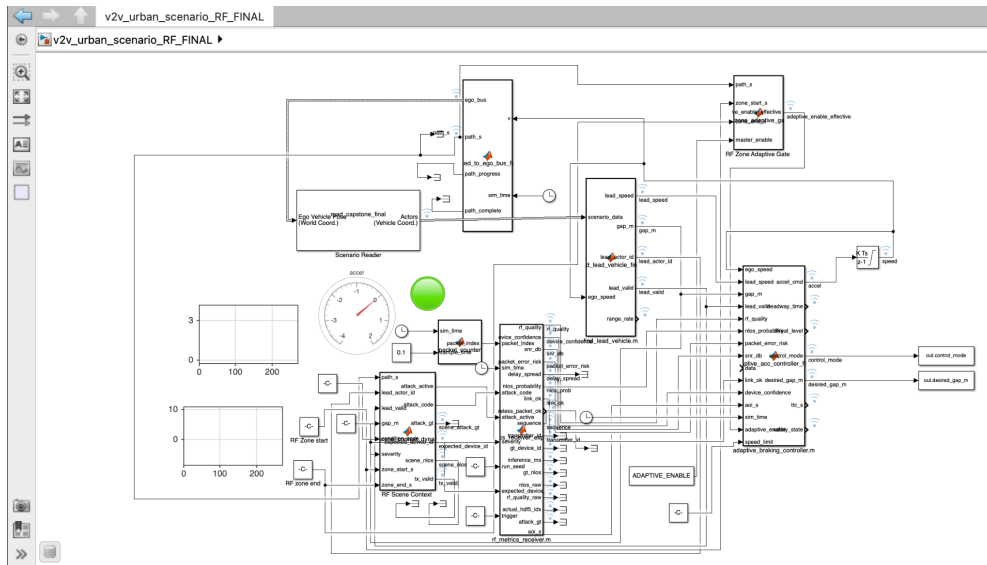


Figure 3.2: Final Simulink implementation with route-based RF context and RF-zone adaptive gating.

3.3 Key Signal Interface

Table 3.1 summarizes the signals exchanged between the physical scene, RF inference layer, and adaptive controller.

Table 3.1: Principal signals crossing the physical, RF, and control layers.

Signal	Role
path_s	Ego route position used to activate the calibrated RF zone.
lead_actor_id	ActorID selected by the dynamic lead tracker.
lead_valid, gap_m	Confirm a physically meaningful V2V peer and its separation.
expected_device_id	Actor-to-RF-class mapping sent to Python.
rf_quality	Normalized communication-quality score supplied to the controller.
nlos_prob	Deep-learning estimate of NLOS probability.
link_ok, sequence	Software-link health and server transaction index.
adaptive_enable_effective	Enables RF-aware adaptation only inside the calibrated route zone.
desired_gap_m	Controller-selected longitudinal target gap.

3.4 Python RF Inference Layer

The Python service receives scene and experiment context, selects a finite I/Q frame, runs the PyTorch model, and returns transmitter prediction, confidence, SNR, packet-error risk, delay spread, NLOS probability, RF quality, packet status, sequence number, and inference latency.

3.5 Fail-Safe Interface Behavior

If Python is unavailable or returns invalid data, the receiver falls back to `link_ok = 0`, `sequence = -1`, RF quality 0.25, and NLOS probability 0.5. Physical gap/TTC control remains active independently of the RF service.

Chapter 4

Dataset and Deep-Learning RF Model

4.1 Finite Synthetic V2V I/Q Dataset

The original synthetic dataset contained 50,000 I/Q frames. A numerical-integrity audit identified 16,732 frames containing NaN or Inf values. These frames were removed before final training, leaving 33,268 finite frames. Each input has shape 2×1024 , representing in-phase and quadrature samples, and the cleaned dataset contains three transmitter classes.

Table 4.1: Dataset integrity summary.

Quantity	Value
Original frames	50,000
Invalid NaN/Inf frames removed	16,732
Finite frames retained	33,268
I/Q shape per frame	2×1024
Transmitter classes	3
NLOS-positive frames ($p_{\text{NLOS}} \geq 0.5$)	11,142

4.2 Preprocessing and Outputs

For each I/Q frame, the pipeline verifies shape and finiteness, removes the I/Q mean, normalizes by RMS magnitude, and verifies the normalized tensor again before inference. The network uses a shared learned representation with task heads for transmitter classification, NLOS classification, and continuous RF-property estimation. The online service converts those outputs into controller-facing quantities such as `rf_quality`, packet-error risk, and NLOS probability. PyTorch was used for training and inference [4].

4.3 Final Training Configuration

4.4 Post-Training Evaluation

The final checkpoint was evaluated on all 33,268 finite frames. The strongest output was NLOS detection; SNR regression was also useful. Transmitter identification and delay-spread regression were substantially weaker and are not used as hard safety gates.

Table 4.2: Final training configuration.

Parameter	Value
Training file	<code>synthetic_v2v_finite.h5</code>
Training / validation sizes	28,278 / 4,990
Epochs	5
Steps per epoch	100
Validation steps	20
Batch size	128
Learning rate	5×10^{-4}
Validation fraction	0.15
Compute device	Apple MPS
Random seed	42
Skipped non-finite batches	0

Table 4.3: Final RF-model evaluation.

Metric	Value	Interpretation
NLOS accuracy	0.9785	Strong overall NLOS classification
NLOS precision	0.9992	Very few false NLOS alarms
NLOS recall	0.9365	Some NLOS frames remain undetected
NLOS balanced accuracy	0.9681	Strong performance across both classes
NLOS F1-score	0.9669	Strong precision–recall balance
NLOS ROC–AUC	0.9999	Excellent ranking capability
SNR MAE	1.831 dB	Useful continuous estimate
SNR R^2	0.8951	Strong explained variance
Device accuracy	0.3542	Insufficient for hard authentication
Device macro-F1	0.3525	Weak across three classes
Delay-spread R^2	−14.18	Not reliable for control
Median inference latency	1.852 ms	Low single-frame latency
p95 inference latency	2.260 ms	Low tail latency after warm-up

Chapter 5

Scene-Conditioned SIL Implementation

5.1 Driving Scenario and RF Interaction Region

The route contains urban intersections, a highway segment, roundabouts, and multiple actors. The final RF experiment is anchored to the ego route rather than to a fixed simulation-time window. Figures 5.1 and 5.2 show two representative operating regions from the final model.

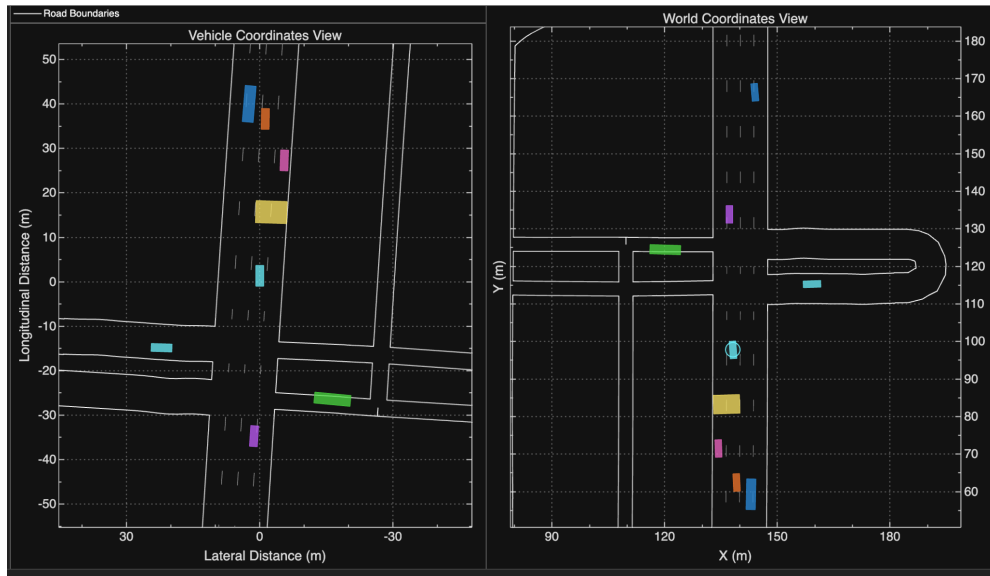


Figure 5.1: Bird's-Eye Scope during the highway segment, showing vehicle- and world-coordinate views.

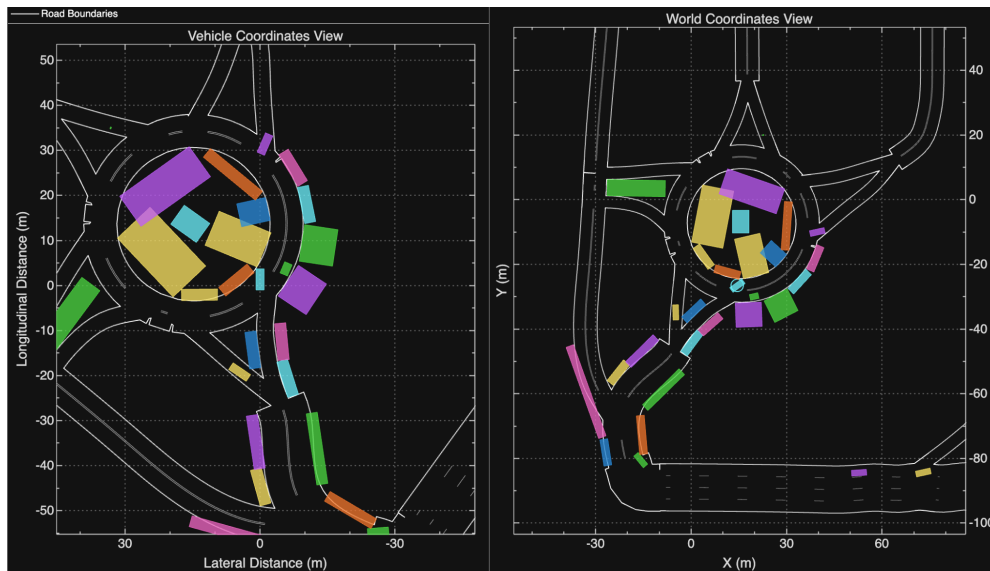


Figure 5.2: Bird's-Eye Scope in the dense roundabout used as the final RF interaction region.

5.2 RF-Zone Calibration

The ego route coordinate `path_s` was logged and mapped to the visually selected urban interaction interval. The final region is

$$959.172 \text{ m} \leq \text{path_s} \leq 1073.437 \text{ m.} \quad (5.1)$$

Lead tracking inside this region showed frequent target changes. The calibration report identified Actor 64 as the longest continuously tracked lead candidate, but only for 1.8 s. The final implementation therefore does not hard-wire one traffic actor as the transmitter; it uses the current valid lead vehicle and maps its ActorID deterministically to one of the three RF device classes.

Command Window			
FINAL SCENE-CONDITIONING PARAMETERS			
RF_ZONE_START_S = 959.172 m			
RF_ZONE_END_S = 1073.437 m			
RF_TX_ACTOR_ID = 64			
Actors detected as valid leads inside the selected zone:			
ActorID	SamplesAsLead	TimeAsLead_s	MinimumGap_m
64	18	1.8	8.4596
72	17	1.7	0.78034
68	10	1	10.823
65	9	0.9	10.423
63	6	0.6	8.556
25	4	0.4	22.804
81	4	0.4	65.204

Figure 5.3: Scene-conditioning calibration and valid-lead audit for the selected RF zone.

5.3 Conditioned I/Q Selection

For each request, the Python service constructs a deterministic selection process from the run seed, packet index, expected device class, and condition code. The clean condition selects a low-NLOS frame for the expected device. The NLOS condition selects a high-NLOS frame for the expected device. Additional jamming and spoofing hooks exist in the service, but they are not used as final validated conditions in this report.

5.4 ZeroMQ Software Bridge

MATLAB and Python communicate on `tcp://127.0.0.1:5557` using a request-reply ZeroMQ bridge. The final bridge uses explicit timeouts, closes failed REQ sockets, recreates them after an exception, and declares returned JSON text as variable-size inside the MATLAB Function block. These changes removed the earlier fixed-size JSON and timeout failures.

```

fprintf('\nRF SMOKE TEST\n');
fprintf('Link success: %.2f %%\n',100*mean(link >= 0.5));
fprintf('Sequence range: %.0f to %.0f\n',min(seq),max(seq));
fprintf('RF-quality range: %.4f to %.4f\n',min(q),max(q));
fprintf('NLOS range: %.4f to %.4f\n',min(nlos),max(nlos));

RF SMOKE TEST
Link success: 100.00 %
Sequence range: 4 to 24
RF-quality range: 0.1432 to 0.9376
NLOS range: 0.0505 to 0.2184
>>

```

(a) Two-second RF smoke test.

```

[REQ] seq=19 pkt=15 idx=4587 pred=0 gt=0 q=0.898 nlos=0.079 wok
=1 lat=1.87ms
[REQ] seq=20 pkt=16 idx=4590 pred=1 gt=0 q=0.896 nlos=0.098 wok
=1 lat=2.01ms
[REQ] seq=21 pkt=17 idx=4599 pred=0 gt=0 q=0.860 nlos=0.115 wok
=1 lat=1.85ms
[REQ] seq=22 pkt=18 idx=4600 pred=2 gt=0 q=0.703 nlos=0.184 wok
=1 lat=1.85ms
[REQ] seq=23 pkt=19 idx=4609 pred=0 gt=0 q=0.674 nlos=0.167 wok
=1 lat=2.01ms
[REQ] seq=24 pkt=20 idx=4610 pred=0 gt=0 q=0.591 nlos=0.169 wok
=1 lat=1.82ms

```

(b) Corresponding Python request log.

Figure 5.4: Verified MATLAB–Python communication with 100% smoke-test link success and nonconstant RF outputs.

5.5 RF-Zone Adaptive Gate

A preliminary clean adaptive run shifted an earlier traffic interaction by approximately 1–1.5 s because small RF-driven acceleration changes accumulated before the intended experiment region. The final model therefore gates RF adaptation with route position:

$$u_{\text{RF}} = u_{\text{master}} \wedge (s_{\text{start}} \leq \text{path_s} \leq s_{\text{end}}) . \quad (5.2)$$

Outside the calibrated zone, the established physical controller determines vehicle motion. Inside the zone, RF-aware adaptation becomes available. Emergency physical gap/TTC logic remains active throughout the route.

5.6 Final Experiment Configuration

The final comparison holds all non-RF settings constant so that the measured difference is attributable to the communication condition rather than to a changed traffic scenario.

Table 5.1: Final matched clean and NLOS configuration.

Setting	Clean	NLOS
Stop time	270 s	270 s
RF trigger / adaptive master	1 / 1	1 / 1
Run seed / speed limit	1 / 10 m/s	1 / 10 m/s
Condition code	0	1
Attack severity	0	0.8
RF zone	959.172–1073.437 m	959.172–1073.437 m

Chapter 6

Experiments, Results and Reliability

6.1 Validation Sequence

The system was validated progressively: signal logging and scene context first, then a short MATLAB–Python smoke test, then a full RF-only NLOS experiment with adaptation disabled, followed by matched 270-second clean and NLOS closed-loop runs.

6.2 RF-Only NLOS Validation

With adaptive control disabled, the complete RF pipeline achieved 100% software-link success across sequence numbers 25–2725 and produced 64 scene-active samples. RF quality dropped sharply inside the selected zone and NLOS probability rose, confirming that scene conditioning was driving the intended I/Q selection.

Table 6.1: RF-only NLOS experiment.

Metric	Value
Software-link success	100.00%
Sequence range	25–2725
Scene-active samples	64
Mean RF quality outside zone	0.7917
Mean RF quality inside zone	0.1155
Mean NLOS probability outside zone	0.1339
Mean NLOS probability inside zone	0.5941
All RF outputs finite	Yes

```
Command Window
FULL RF-ONLY NLOS RESULT
Successful link samples: 100.00 %
Sequence range: 25 to 2725
Scene-active samples: 64
Mean RF quality outside zone: 0.7917
Mean RF quality inside zone: 0.1155
Mean NLOS outside zone: 0.1339
Mean NLOS inside zone: 0.5941
All RF outputs finite: 1
>>
```

Figure 6.1: Command-window audit of the full RF-only NLOS run.

6.3 Final Clean-versus-NLOS Closed-Loop Comparison

Both final runs maintained 100% MATLAB–Python software-link success. The NLOS run reduced mean in-zone RF quality by 0.2615 and increased mean predicted NLOS probability by 0.1763. Minimum speed and peak deceleration were unchanged, showing that the RF experiment did not create a new hard-braking limit. The route-aligned speed response changes only modestly because the RF layer is supervisory and the physical traffic-following controller remains dominant.

Table 6.2: Final corrected route-aligned comparison.

Run	Link (%)	Mean RF-q (zone)	Mean NLOS (zone)	Min speed (m/s)	Peak decel. (m/s ²)
Clean	100.00	0.7959	0.1330	0.000	−7.500
NLOS	100.00	0.5343	0.3093	0.000	−7.500



Figure 6.2: Final corrected clean-versus-NLOS comparison aligned by ego-route position.

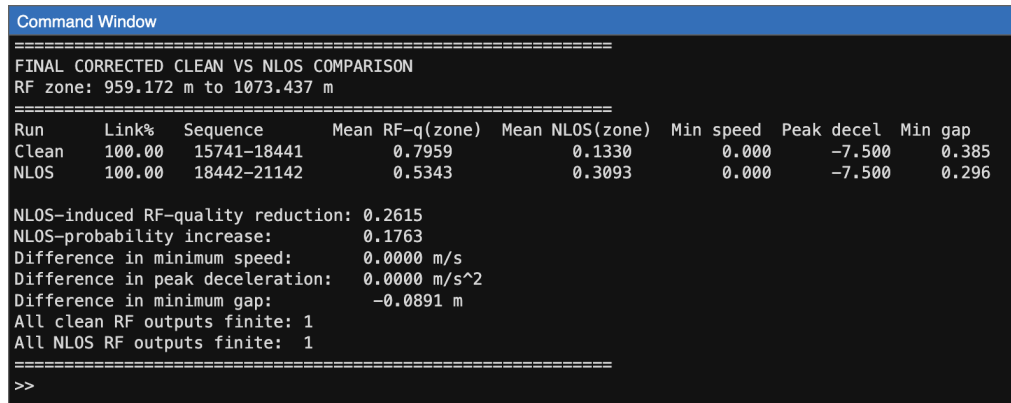


Figure 6.3: Command-window audit of the final corrected clean and NLOS runs.

6.4 Security and Reliability Assessment

The final evidence supports successful SIL integration: the scene activates NLOS behavior, the software link remains available, learned NLOS probability changes in the expected direction, and the longitudinal response remains bounded. Device identification is not accurate enough for a hard authentication gate, delay-spread regression is not reliable enough for safety-critical control, and the raw minimum-gap signal is not used as standalone collision evidence because dense-roundabout target switching can affect it.

Table 6.3: Validated capabilities and current boundaries.

Capability	Evidence	Status
Scene-conditioned NLOS activation	64 active samples with strong RF-only inside/outside contrast	Validated
MATLAB–Python SIL link	100% link success in smoke and final runs	Validated
Low-latency NLOS inference	1.85-ms median, 2.26-ms p95	Validated
Stable RF-aware vehicle control	Same peak deceleration with bounded route-aligned speed change	Validated
Hard transmitter authentication	Device macro-F1 = 0.353	Limitation
Delay-spread control input	Delay-spread $R^2 = -14.18$	Limitation
Geometry-derived RF propagation	Dataset-conditioned replay; no ray tracing	Limitation
Independent external benchmark	Full-dataset post-training evaluation	Limitation

Clean and NLOS are the two fully reported final conditions. Jamming and spoofing remain implemented experiment hooks and require dedicated multi-seed campaigns before equivalent performance or security claims are made.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project developed a complete V2V RF-intelligence and adaptive longitudinal-control research prototype. MATLAB/Simulink provides the physical driving scenario, ego-path state, dynamic lead tracking, and vehicle controller; Python/PyTorch provides real-time RF inference; and ZeroMQ links both environments during simulation. The final route-based scene context activates RF behavior only in the calibrated urban region, while a separate adaptive gate preserves the established physical trajectory elsewhere.

The final RF model showed strong NLOS discrimination ($F1 = 0.9669$, balanced accuracy = 0.9681, ROC-AUC = 0.9999) with 1.85-ms median and 2.26-ms p95 single-frame inference latency. The RF-only experiment produced a clear in-zone degradation signature, and both final 270-second clean and NLOS runs maintained 100% MATLAB–Python link success. In the calibrated zone, mean RF quality changed from 0.7959 to 0.5343 and mean predicted NLOS probability from 0.1330 to 0.3093. The resulting control response remained bounded because RF information was supervisory rather than a replacement for physical safety logic.

The system should therefore be viewed as a production-oriented **scene-conditioned RF software-in-the-loop prototype**: reproducible, fail-safe at the software interface, quantitatively validated for clean/NLOS experiments, but not yet a certified controller or a geometry-based RF digital twin.

7.2 Future Work

- **Independent RF benchmarking:** reserve source-disjoint generation seeds and impairment profiles for a true test set and repeat results over multiple training seeds.
- **Stronger transmitter verification:** improve class balancing, channel-invariant embeddings, confidence calibration, and open-set rejection before device identity is used for security decisions.
- **Geometry- or hardware-based RF:** replace dataset-conditioned replay with ray tracing, measured channel impulse responses, or SDR-generated over-the-air I/Q streams.
- **Expanded closed-loop campaigns:** validate jamming and spoofing across multiple seeds using collision/TTC, headway, jerk, packet delivery, and age-of-information metrics.

Appendix A

Reproduction Commands

1. Train the final synthetic-domain model

```
python -u -m src.level4_fingerprinting.train \  
  --hdf5 data/processed/synthetic_v2v_finite.h5 \  
  --out-dir results/checkpoints_synthetic_correct \  
  --epochs 5 --steps-per-epoch 100 --validation-steps 20 \  
  --batch-size 128 --lr 0.0005 --val-fraction 0.15 \  
  --device mps --log-every 20 --seed 42
```

2. Evaluate the checkpoint

```
KMP_DUPLICATE_LIB_OK=TRUE python -u evaluate_rf_model_research.py \  
  --model results/checkpoints_synthetic_correct/best.pt \  
  --hdf5 data/processed/synthetic_v2v_finite.h5 \  
  --device cpu --batch-size 128 --max-samples 0 \  
  --out-dir results/rf_model_evaluation_synthetic_correct
```

3. Start the experiment inference service

```
KMP_DUPLICATE_LIB_OK=TRUE PYTHONPATH=. python -u \  
  src/level4_fingerprinting/infer_service_experiments.py \  
  --model results/checkpoints_synthetic_correct/best.pt \  
  --hdf5 data/processed/synthetic_v2v_finite.h5 \  
  --port 5557 --device cpu --log-every 20
```

4. MATLAB startup and final run parameters

```
prepare_ego_path  
load('rf_scene_calibration.mat')  
mdl = 'v2v_urban_scenario_RF_FINAL';  
RF_TRIGGER = 1; ADAPTIVE_ENABLE = 1;  
RUN_SEED = 1; SPEED_LIMIT_MPS = 10;  
set_param(mdl, 'StopTime', '270');  
  
% Clean: CONDITION_CODE = 0; ATTACK_SEVERITY = 0;  
% NLOS : CONDITION_CODE = 1; ATTACK_SEVERITY = 0.8;  
clear v2v_zmq_request_experiment  
out = sim(mdl);
```

Bibliography

- [1] K. Sankhe, M. Belgiovine, F. Zhou, S. Riyaz, S. Ioannidis, and K. R. Chowdhury, “ORACLE: Optimized Radio clAssification through Convolutional neural nEtworks,” in *Proc. IEEE INFOCOM*, 2019, pp. 370–378, doi: 10.1109/INFOCOM.2019.8737463. Also available as arXiv:1812.01124.
- [2] M. Hasan, S. Mohan, T. Shimizu, and H. Lu, “Securing Vehicle-to-Everything (V2X) Communication Platforms,” *IEEE Transactions on Intelligent Vehicles*, vol. 5, no. 4, pp. 693–713, Dec. 2020, doi: 10.1109/TIV.2020.2987430.
- [3] T. J. O’Shea and J. Hoydis, “An Introduction to Deep Learning for the Physical Layer,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, 2017, doi: 10.1109/TCCN.2017.2758370.
- [4] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [5] P. Hintjens, *ZeroMQ: Messaging for Many Applications*. Sebastopol, CA, USA: O’Reilly Media, 2013.